Getting Started with the QARG Python Training



School of Chemistry and Physics
Astrophysics Research Group

This guide is for you if you're new to Python, or haven't used it in a while. Read it before you open Part 1. It takes about 15 minutes, and it'll save you a lot of confusion later.

Contents

1. Words you'll meet
2. Your first session in Google Colab
3. Working in a notebook
4. How to work through the training
5. When something goes wrong
6. Asking for help
7. Setting up Python on your own computer
8. Python at a glance

1. Words You'll Meet

Word	What it means
Python	The programming language. It's the most widely used language in astronomy.
Notebook	A document that mixes text, code and results (plots, tables, numbers). Notebook files end in `.ipynb`.
Cell	One block in a notebook. **Text cells** explain things. **Code cells** contain Python you can run.
Jupyter	The software that runs notebooks. *JupyterLab* is its interface on your own computer.
Google Colab	A free Google website that runs Jupyter notebooks in your browser, so you don't have to install anything.
Kernel or **runtime**	The Python program running behind the notebook. It remembers everything you've run so far.
Library or **package**	Code other people have written that you can use, like NumPy (numbers), pandas (tables), Matplotlib (plots) and Astropy (astronomy).
Environment	A self-contained installation of Python and its packages on your computer, so different projects don't interfere with each other.
Terminal	A window where you type commands instead of clicking. Also called the command line, shell or prompt.
GitHub	A website for storing and sharing code. This training lives there.

Word	What it means
Repository (repo)	A project folder on GitHub, with its full history of changes.

2. Your First Session in Google Colab

You'll need a Google account and a modern web browser (Chrome, Firefox, Edge or Safari).

1. **Open the README** at https://github.com/mjcowley/Python_Intro and click the **Open in Colab** button next to Part 1.
2. **Sign in** to Google if Colab asks you to.
3. **Save your own copy.** Go to *File → Save a copy in Drive*. A new tab opens with a copy named "Copy of 01-Python-Basics.ipynb". **Work in this copy.** Anything you do in the original isn't saved.
4. **Run the first code cell.** Click inside it and press **Shift + Enter**. The first time, Colab takes a few seconds to start up, and may warn you that the notebook was not written by Google. Click **Run anyway**: it comes from this training.
5. **Keep going** down the notebook, reading each text cell and running each code cell in order.

Finding your work again. Your copies are saved in a folder called *Colab Notebooks* in your Google Drive. You can also go to https://colab.research.google.com, choose *File → Open notebook*, and pick the *Google Drive* tab.

Colab disconnects if you leave it idle for a while, and every session has a time limit. Your notebook stays saved, but the kernel forgets everything you'd run. When that happens, see Section 5.

3. Working in a Notebook

Running cells

- **Shift + Enter** runs the current cell and moves to the next one. This is the one you'll use most.
- **Ctrl + Enter** (**Cmd + Enter** on a Mac) runs the cell and stays on it, which is handy when you're tweaking something.
- The **▶ button** to the left of a code cell does the same thing with the mouse.

The marker next to a code cell tells you its state:

Marker	Meaning
`[ ]` (empty)	This cell hasn't been run since the kernel started.
`[*]` or a spinning icon	The cell is running. Wait for it to finish.
`[5]`	This cell was the 5th one run. The numbers show the order things actually happened.

The one rule that causes most confusion

The kernel only knows about cells you have run, in the order you ran them. It doesn't care where a cell sits on the page. If you skip a cell that creates a variable, every later cell that uses it fails. If you go back and change a cell, the cells below it don't update until you run them again.

When things get confusing, start fresh: *Runtime → Restart session*, then *Runtime → Run before* from the cell you were working on.

Cells that stop on purpose. Colab stops running as soon as a cell fails. Part 1 has three cells in its *Errors and Debugging* section that fail deliberately, so *Run all* and *Run before* stop at each of them. That's expected: click the cell after the one that failed and choose *Runtime → Run after* to carry on. Check cells for exercises you haven't done yet don't stop anything. They just print "Not done yet".

Useful Colab shortcuts

On a Mac, use **Cmd** wherever this says **Ctrl**. For the two-step shortcuts, press **Ctrl + M**, let go, then press the letter.

Shortcut	What it does
Shift + Enter	Run the cell and move to the next
Ctrl + Enter	Run the cell and stay on it
Ctrl + M, then B	Add a code cell below
Ctrl + M, then A	Add a code cell above
Ctrl + M, then D	Delete the cell
Ctrl + /	Comment or uncomment the selected lines
Ctrl + Z	Undo typing in a cell
Tab	Complete the name you're typing (variables, functions, columns)

You can also add cells with the **+ Code** and **+ Text** buttons at the top of the page, and open the **Table of contents** from the ☰ icon on the left to jump between sections.

Getting help on a function

You don't need to memorise how every function works. To see its documentation:

- Type the function name followed by `?` in a code cell and run it, for example `np.linspace?`
- Or run `help(np.linspace)`
- In Colab, pausing after typing an opening bracket, like `np.linspace(`, shows a pop-up with its arguments.

The official documentation for every library has examples too. Searching for "numpy linspace" or "pandas merge" usually finds the right page first.

4. How to Work Through the Training

The training has six parts and takes about 17 hours in total. Most people spread it over two to three days, and some prefer an hour or two a day over a couple of weeks. Do the parts in order, since each builds on the ones before it.

Tips that make a real difference

- **Type the code yourself.** It feels slower than reading, but it's how the syntax sticks. Copy-pasting teaches you very little.
- **Experiment.** Change a number in a worked example and re-run it. Try to break it on purpose and read the error. You can't damage anything, and you can always reopen a fresh copy.
- **Have a real go at each exercise** before opening the solution. Use the check cells to test your answer. Being stuck for ten minutes is normal and useful. Being stuck for an hour isn't, so open the solution or ask.
- **Read the solutions even when you got it right.** There's usually more than one way to do something, and the solution may show a shortcut.
- **Take breaks.** Stop at the end of a section, not halfway through one.
- **Keep notes** of things you want to remember or ask about. A text cell at the top of your copy works well.

About AI tools. Assistants like ChatGPT, Copilot and Claude are useful for explaining errors and concepts, and you'll probably use them in your project. For this training, try each exercise yourself first. The point is to be able to read, check and fix code, including code an AI writes for you.

5. When Something Goes Wrong

Errors are a normal part of programming. Everyone gets them all the time. **Read the error message from the bottom up**: the last line says what went wrong, and the lines above show where.

What you see	What it usually means	What to do
`NameError: name 'x' is not defined`	You're using a variable the kernel hasn't seen, usually because a cell above wasn't run, or a typo	Run the cells above (*Runtime → Run before*), and check the spelling. Python cares about capital letters.
`ModuleNotFoundError: No module named '...'`	The library isn't installed or wasn't imported	Run the notebook's **Setup** cell. On your own computer, install it with `pip install <name>` in your environment.
`SyntaxError` or `IndentationError`	The code isn't valid Python: a missing bracket, quote, colon, or mismatched spaces	Look at the line the arrow points to, and the line just above it.

What you see	What it usually means	What to do
`TypeError`	An operation got the wrong kind of value, like adding text to a number	Use `print(type(x))` to check what your variables actually are.
`KeyError: 'column'`	That column or dictionary key doesn't exist	Print `df.columns` to see the real names. Check spelling and capitals.
A check cell prints "Not done yet" or "Not quite"	The exercise above it isn't finished, or the answer isn't right yet	Read the message for a hint, fix your answer, and run the check cell again.
A cell shows `[*]` for a long time	It's still running, or it's stuck	Wait a little. If it's stuck, use *Runtime → Interrupt execution*.
"Runtime disconnected" or everything says "not defined"	Colab disconnected and the kernel forgot your variables	Reconnect, then *Runtime → Run before* from where you were.
A data download fails	A network problem, or the file isn't available	Check your internet connection, then re-run the cell. If it keeps failing, ask in the group.
Results look wrong or inconsistent	Cells were run out of order, or something was changed and not re-run	*Runtime → Restart session and run all*.
You've broken your copy and want to start again	It happens to everyone	Reopen the original with the **Open in Colab** button and save a new copy.

Search the error message. Copy the last line of the error (leave out your own variable names) into a search engine. Someone has almost always had the same problem, and Stack Overflow answers are usually near the top.

6. Asking for Help

Asking for help is a skill, and asking early is a good habit, not a weakness. A good rule: if you've been stuck for about 30 minutes and searching hasn't helped, ask.

You'll get a faster, better answer if you include:

1. **What you were trying to do.** For example: "Select galaxies with $1 < z < 2$ in Part 3, Section 2."
2. **What you ran.** Paste the code as text, not a screenshot.
3. **The full error message**, including the traceback.
4. **What you've already tried.**

Often, writing this out helps you spot the problem yourself.

Where to ask: others doing the training, the rest of the group, and your supervisor. Group resources are on the QARG Canvas site.

7. Setting Up Python on Your Own Computer

You don't need this for the training, since Colab works fine. But once your project starts, you'll want Python on your own machine. We use **Miniforge**, a free, lightweight installer for Python and the `conda` package manager.

Step 1: Install Miniforge

Windows

1. Go to the Miniforge download page and download the Windows installer (`Miniforge3-Windows-x86_64.exe`).
2. Run it and accept the defaults. Install it "Just for me", not for all users.
3. From the Start menu, open **Miniforge Prompt**. This is the terminal you'll use for everything below.

macOS

1. Open **Terminal** (in *Applications → Utilities*, or search for it with Cmd + Space).
2. Copy and paste these two commands, pressing Enter after each:

```
curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
```

3. Press Enter to read the licence, type `yes` to accept it, accept the default location, and type `yes` when it asks whether to initialise conda.
4. **Close Terminal and open a new one.** Your prompt should now start with `(base)`.

Linux

Follow the macOS instructions in your usual terminal.

Step 2: Create an environment for your work

In your terminal (Miniforge Prompt on Windows), run:

```
conda create -n qarg python=3.12
conda activate qarg
```

Answer `y` if asked to proceed. Your prompt now starts with `(qarg)`, which means you're in the new environment. Anything you install from now on goes into it.

Step 3: Download the training and install the packages

1. On the GitHub page, click the green **Code** button, then **Download ZIP**, and unzip it somewhere you'll find it, such as your Documents folder. (If you already use git, `git clone https://github.com/mjcowley/Python_Intro.git` does the same thing.)

2. In your terminal, move into that folder with `cd`, for example:

```
cd Documents/Python_Intro
```

3. Install the packages and start JupyterLab:

```
pip install -r requirements.txt
jupyter lab
```

JupyterLab opens in your web browser. Double-click a notebook in the file list on the left to open it. It works just like Colab, except the files are on your computer and saved automatically.

Every time you come back

```
conda activate qarg
cd Documents/Python_Intro
jupyter lab
```

To stop JupyterLab, close the browser tab, then press **Ctrl + C** in the terminal.

If something doesn't work

- **"conda is not recognised" or "command not found"**: on Windows, make sure you're in *Miniforge Prompt*, not the ordinary Command Prompt. On macOS and Linux, close the terminal and open a new one.
- **A package won't import in JupyterLab**: check your prompt says `(qarg)` before you start JupyterLab.
- **Still stuck?** Ask in the group. Setup problems are very common, and someone will have seen yours before.

If you'd rather use an editor, VS Code and VSCodium both run notebooks well. Install their Python and Jupyter extensions, and choose the `qarg` environment when they ask which Python to use.

8. Python at a Glance

A quick reminder of syntax covered in the training. Keep it next to you while you work.

Basics

```
x = 3.5                          # create a variable
name = "NGC 1300"                # a string (text)
print(f"{name} is at z = {x:.2f}")   # f-string: values inside {}, :.2f = 2 decimal places
# anything after a hash is a comment
```

Lists and dictionaries

```
z = [0.5, 1.2, 2.1]               # a list
z[0], z[-1], z[1:3]               # first, last, a slice (index 1 and 2)
z.append(3.0)                     # add to the end
galaxy = {"name": "A", "z": 1.2}  # a dictionary
galaxy["z"]                       # look up a value by its key
```

Decisions and loops

```
if z_gal > 1:
    print("high")                 # indented lines belong to the if
elif z_gal > 0.5:
    print("medium")
else:
    print("low")

for value in z:                   # repeat for each item
    print(value)

high = [v for v in z if v > 1]    # list comprehension
```

Functions

```
def distance_modulus(d_mpc):
    """Distance modulus for a distance in Mpc."""
    return 5 * np.log10(d_mpc) + 25
```

NumPy and Astropy

```
import numpy as np
import astropy.units as u

a = np.array([1.0, 2.0, 3.0])
a * 2, np.log10(a), np.mean(a)    # maths works on every element at once
a[a > 1.5]                        # boolean mask: keep elements where True
mask = (a > 1) & (a < 3)          # combine conditions with & | ~ and brackets
d = (16.4 * u.Mpc).to(u.lyr)      # units and conversions
```

pandas tables

```
import pandas as pd

df = pd.read_csv("file.csv", na_values=[-99])   # load a table, -99 means missing
df.head(), df.info(), df.describe()             # look at it
df["z"]                                         # one column
df[df["z"] > 1]                                 # rows matching a condition
df["mag"] = 25 - 2.5 * np.log10(df["flux"])     # a new column
merged = pd.merge(df1, df2, on="id")            # combine tables on a shared ID
```

Plotting

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.scatter(x, y, s=5, alpha=0.5, label="Galaxies")
ax.set_xlabel("Redshift")
ax.set_ylabel(r"$\log_{10}(M_*/M_\odot)$")      # r"..." for LaTeX labels
ax.legend()
fig.savefig("figure.pdf", bbox_inches="tight")
plt.show()
```

Good luck, and welcome to the group!